Curso SQL

Comisión 81810
Alumno: Nicolás Pereyra

Scooter rental business database

Introducción

El presente proyecto desarrolla el diseño y la estructura de una base de datos relacional para una startup de movilidad sostenible que se especializa en el alquiler de monopatines eléctricos en zonas metropolitanas. El sistema busca centralizar el control operativo de la flota y el registro transaccional de los usuarios. Esta infraestructura de datos es fundamental para transformar estos registros brutos en indicadores de rendimiento (KPIs), permitiendo a la organización evaluar su estado actual y realizar proyecciones de rentabilidad basadas en datos históricos de consumo y operatividad.

Situación Problemática

Actualmente, la empresa carece de un sistema integrado que permita rastrear la trazabilidad de los viajes y la usabilidad de los activos (scooters). Esta desorganización genera brechas en el análisis de la clientela y en el control financiero. La implementación de esta base de datos resuelve la necesidad de estructurar métricas críticas como la frecuencia de uso por estación, el flujo de ingresos por métodos de pago y la gestión de egresos operativos, permitiendo así una visión clara de la rentabilidad del modelo de negocio.

Objetivo:

Centralizar la información operativa y transaccional para optimizar la toma de decisiones en tres áreas clave:

- Logística: Medir la usabilidad de las estaciones y el estado de la flota (batería y mantenimiento).
- Analítica: Identificar patrones de uso (frecuencia, horarios pico y rutas comunes).
- Contable: Controlar la facturación, los montos recaudados por periodo y la efectividad de los métodos de pago.

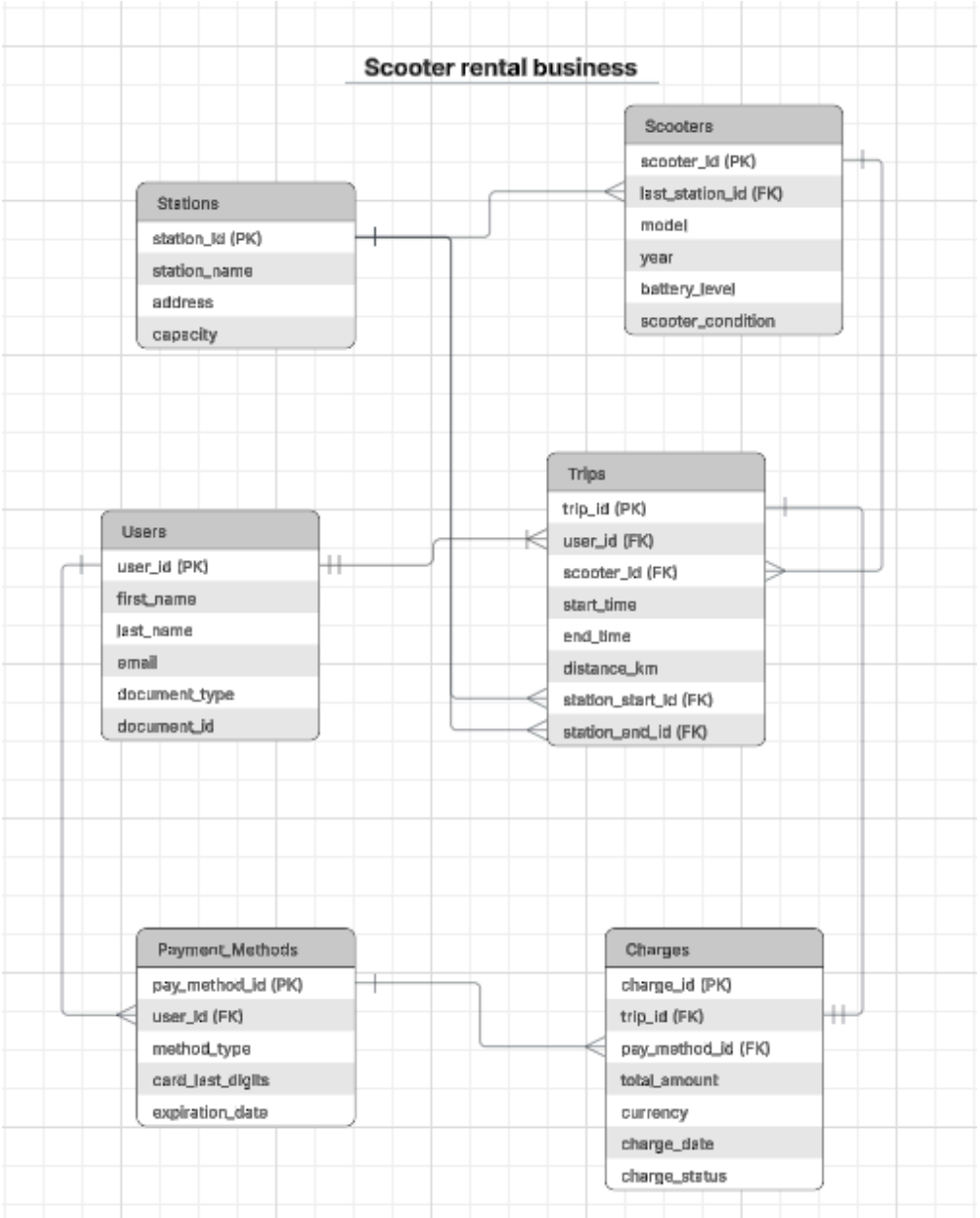
Tablas y tipos de datos

Tabla	Descripción	Campos	Tipo de Dato
Stations	Ubicaciones	station_id (PK) / name / address / capacity	INT / VARCHAR / VARCHAR / INT

	físicas		
Users	Clientes registrados	user_id (PK) / first_name / last_name / email (unique) / document_type / document	INT / VARCHAR / VARCHAR / VARCHAR / VARCHAR / INT
Scooters	Flota de vehículos	scooter_id (PK) / last_station_id (FK) / model / year / battery_level / scooter_condition	INT / INT / VARCHAR / INT / DECIMAL / INT / VARCHAR
Trips	Viajes realizados	trip_id (PK) / user_id (FK) / scooter_id (FK) / station_start_id (FK) / station_end_id (FK) / start_time / end_time / distance_km	INT / INT / INT / INT / INT / DATETIME / DATETIME / DECIMAL
Payment Methods	Método de pagos	pay_method_id (PK) / user_id (FK) / method_type / card_last_digits / expiration_date	INT / INT / VARCHAR / INT / DATE
Charges	Facturación	charge_id (PK) / trip_id (FK) / pay_method_id (FK) / total_amount / currency / charge_date / charge_status	INT / INT / INT / DECIMAL / VARCHAR / DATETIME / VARCHAR

Relaciones entre tablas:

- Station 1 a N Scooters (Para cada estación habrá muchos scooters estacionados para utilizar)
- Station 1 a Trips (Una estación puede formar parte de muchos viajes)
- Scooters 1 a N a Trips (Un monopatín puede ser utilizado en muchos viajes)
- Users 1 a N a Trips (Un usuario puede hacer muchos viajes)
- Users 1 a N a Payment Methods (Un usuario puede utilizar varios métodos de pago)
- Charges 1 a 1 Trips (Habrá un solo cargo por viaje)
- Payment Methods 1 a N Charges (Un método de pago se usa para muchos cargos)



Importación de Datos (ETL) - Scooter Rental Business Database

Introducción

Para la carga de datos en las tablas, se eligió la importación en formato CSV. Se priorizó el orden jerárquico de las tablas (relación Padre - Hijo) para asegurar la correcta integridad referencial durante el proceso.

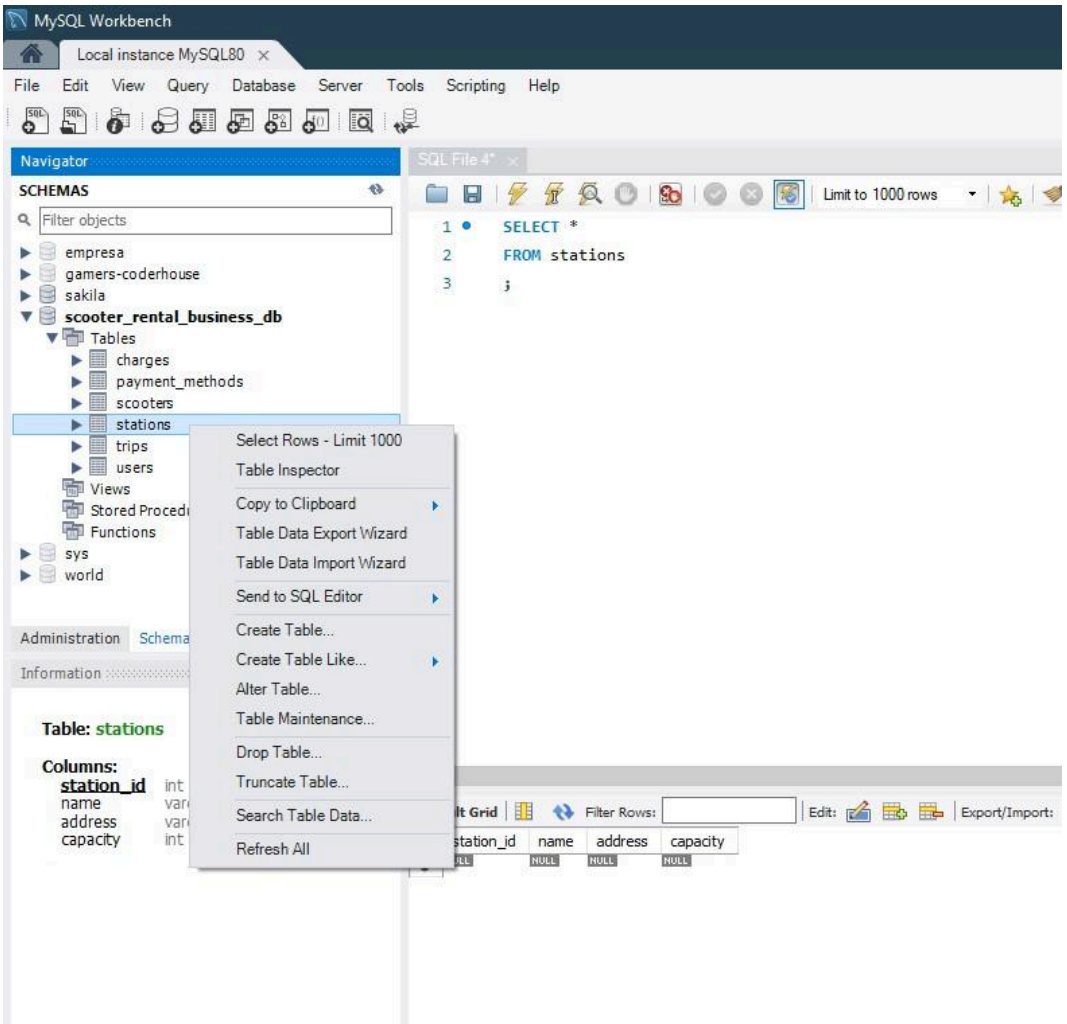
Orden de Importación

El orden de importación fue estricto debido a las restricciones impuestas por las *Foreign Keys* (*constraints*). El orden de inserción de datos fue:

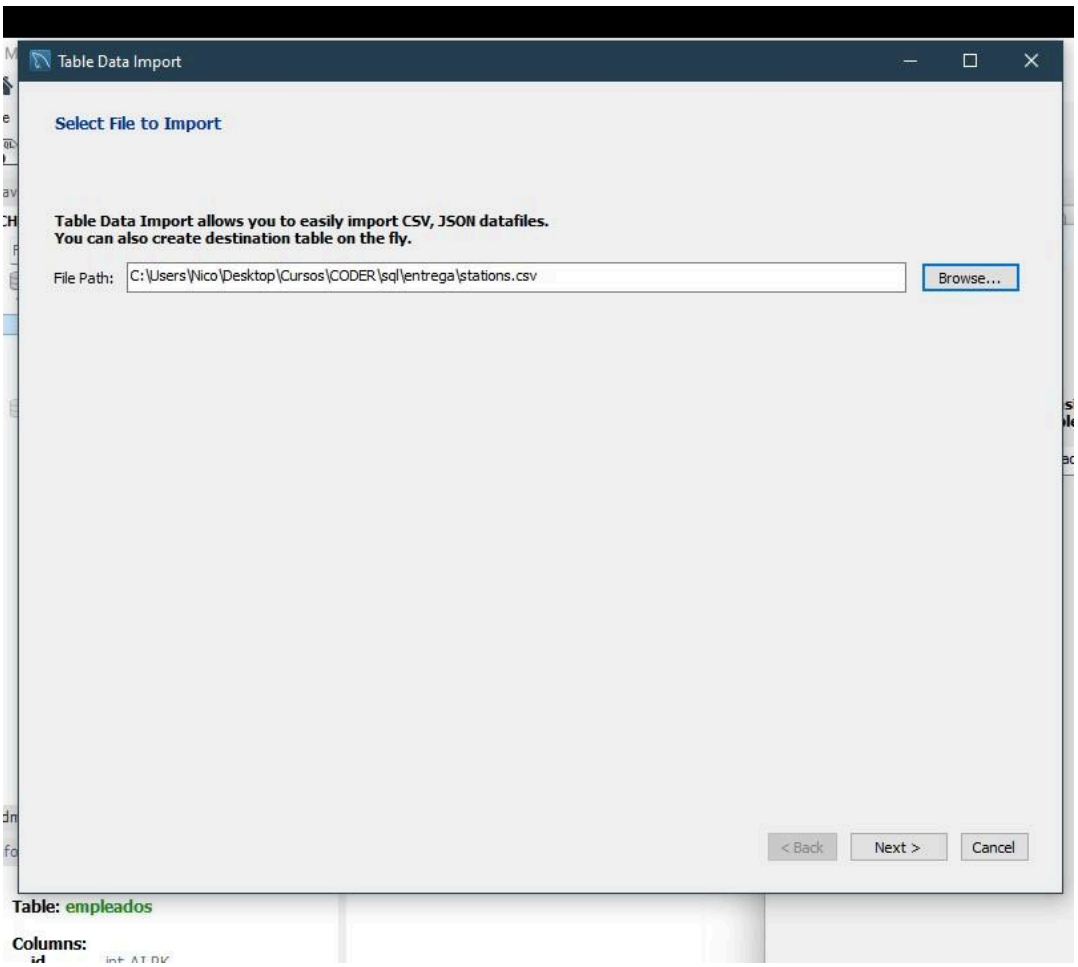
1. `Stations`
2. `Users`
3. `Scooters`
4. `Payment_Methods`
5. `Trips`
6. `Charges`

Paso a paso para importar cada archivo:

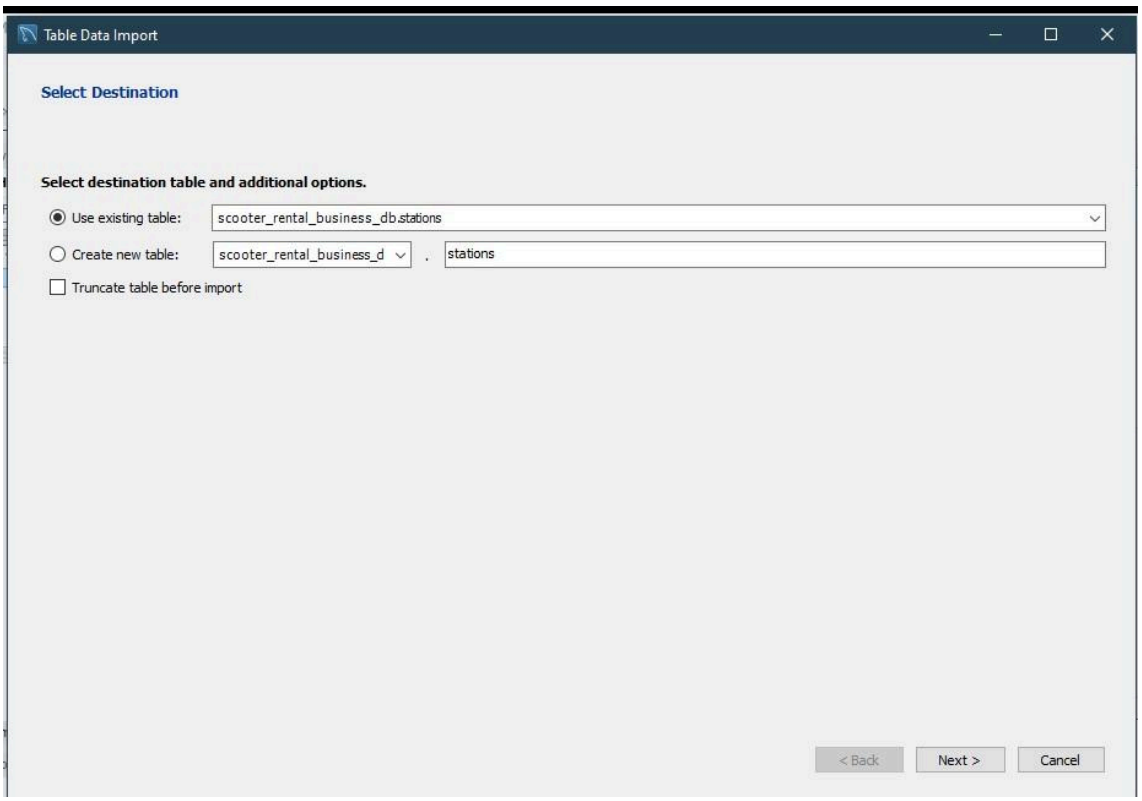
1. **Preparación del entorno:** Nos aseguramos de haber ejecutado el script de creación de tablas (`CREATE TABLE`) y que las tablas estén vacías.



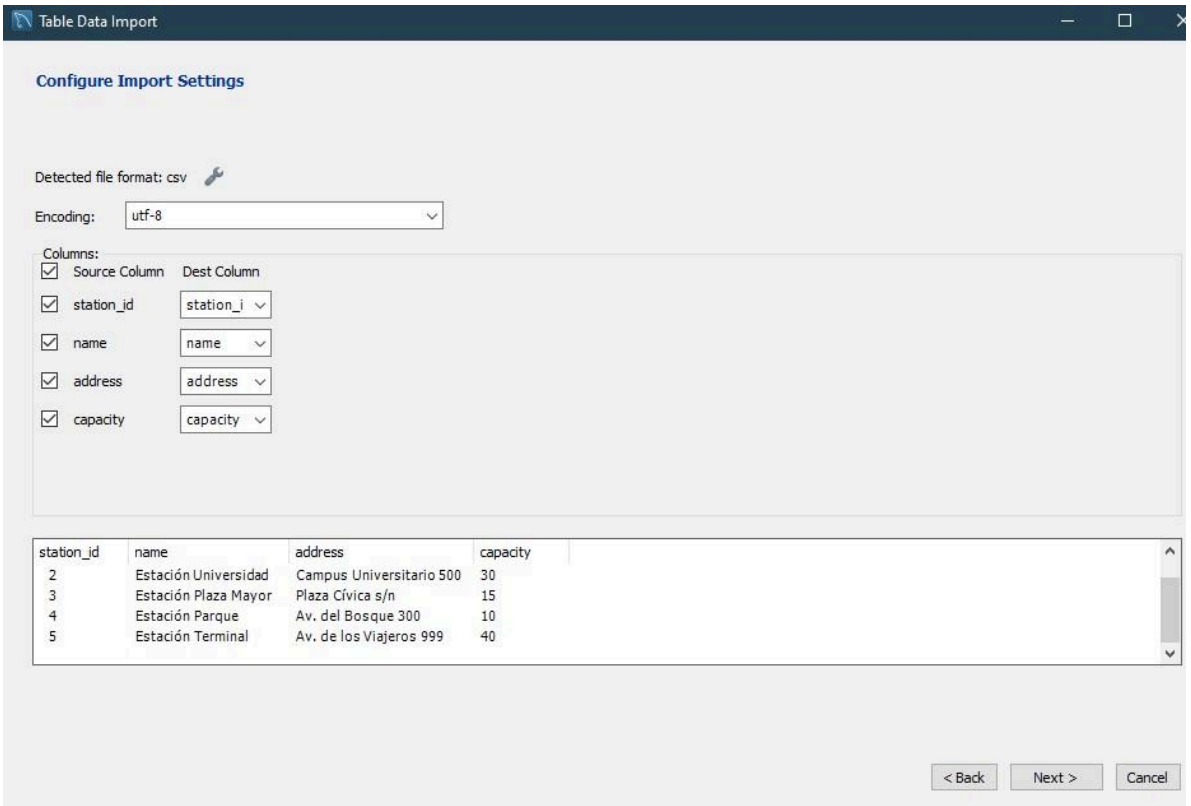
2. **Abrir el Asistente:** En el panel de navegación izquierdo (Schemas) de MySQL Workbench, se despliega la base de datos y hacer clic derecho sobre la tabla que se desea insertar los datos (ej. `Stations`).
3. **Seleccionar la herramienta de importación:** En el menú desplegable, seleccionar "Table Data Import Wizard".



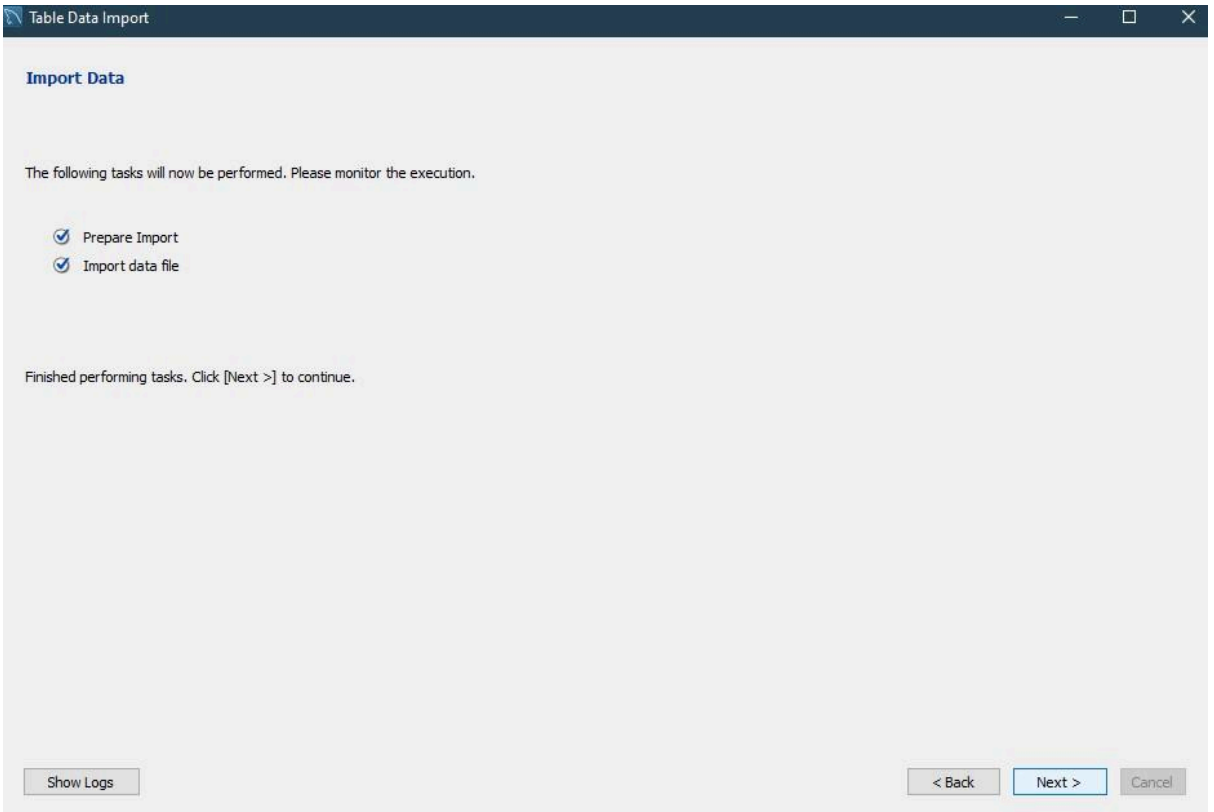
4. **Cargar el archivo:** Se abrirá una ventana. Hacer clic en "Browse" y buscar el archivo `.csv` correspondiente en la computadora (ej. `stations.csv`). Hacer clic en "Next".
5. **Configurar el destino:** Seleccionar la opción "Use existing table" (Usar tabla existente) en este caso y verificar que el nombre de la tabla seleccionada sea el correcto. Hacer clic en "Next".



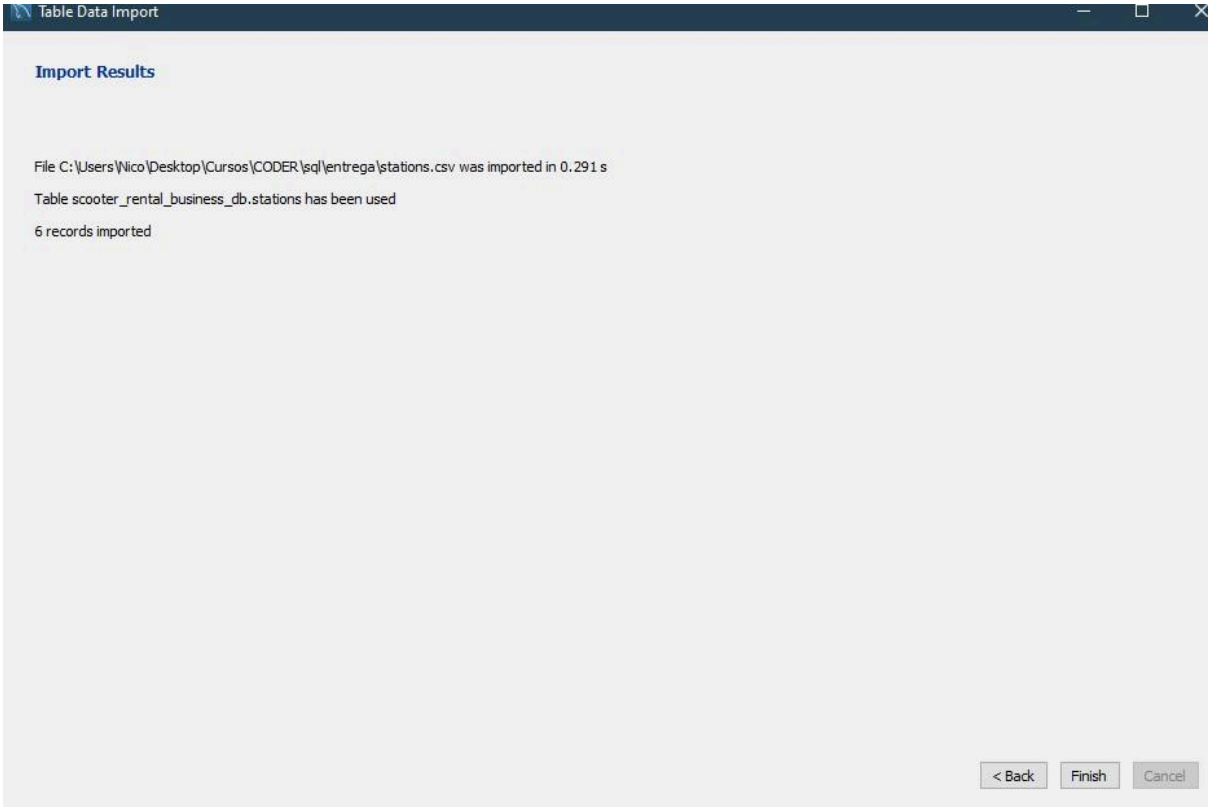
6. **Mapeo de Columnas:** En la pantalla de configuración, verificar que las columnas del archivo origen (Source Column) coincidan con las columnas de destino (Field) de la tabla en MySQL. Como los archivos CSV tienen los mismos nombres en la cabecera, el mapeo debería ser automático. Hacer clic en "Next".



7. **Ejecutar Importación:** Hacer clic en "Next" para comenzar la ejecución de las tareas. MySQL leerá el archivo e insertará los registros uno por uno.



8. **Finalizar:** Una vez que aparezca el mensaje de éxito, hacer clic en "Finish".



9. **Verificar:** Dentro de una hoja de SQL de nuestra DB ejecutar un `SELECT * FROM nombre_de_la_tabla;` para comprobar que los datos se cargaron correctamente.

station_id	name	address	capacity
1	Estación Central	Av. Siempre Viva 123	20
2	Estación Universidad	Campus Universitario 500	30
3	Estación Plaza Mayor	Plaza Cívica s/n	15
4	Estación Parque	Av. del Bosque 300	10
5	Estación Terminal	Av. de los Viajeros 999	40
6	Estación Shopping	Centro Comercial 100	25
NULL	NULL	NULL	NULL

10. **Repetir:** Se repite los pasos del 2 al 9 para el resto de los archivos respetando el orden jerárquico mencionado al principio.

VIEWS

#1 Ver listado de solo la flota de scooters en condición óptima disponibles.

```
CREATE VIEW vw_flota_optima AS
SELECT
        sc.scooter_id,
   st.name as current_station,
   sc.battery_level,
   sc.scooter_condition
FROM scooters sc
INNER JOIN stations st
        ON sc.last_station_id = st.station_id
WHERE sc.scooter_condition = 'Óptimo'
ORDER BY
        sc.battery_level DESC;
```

```
SELECT *
FROM vw_flota_optima;
```

TABLAS UTILIZADAS: scooters, stations.

#2 Ver detalle descriptivo del usuario que realiza un viaje, esto nos sirve como registro detallado de la clientela.

```
CREATE VIEW vw_detalle_viajes AS
SELECT
        t.trip_id,
        u.first_name as nombre,
   u.last_name as apellido,
   s.model as modelo_scooter,
   s2.name as estacion_partida,
   s3.name as estacion_llegada,
   CAST(t.end_time AS DATE) as trip_date,
   t.distance_km as kilometros_recorridos
FROM trips t
INNER JOIN users u
        ON t.user_id = u.user_id
INNER JOIN scooters s
        ON t.scooter_id = s.scooter_id
INNER JOIN stations s2
        ON t.station_start_id = s2.station_id
INNER JOIN stations s3
        ON t.station_end_id = s3.station_id

ORDER BY
        trip_date, trip_id ASC
;

SELECT *
FROM vw_detalle_viajes;
```

TABLAS UTILIZADAS: users, scooters, stations.

#3 Ver el flujo de ingresos de la empresa de forma histórica estilo KPIs

```
CREATE VIEW vw_ingresos_ft_metododepago AS
SELECT
        p.method_type as Metodo_pago,
   sum(c.total_amount) as Monto_total
FROM payment_methods p
INNER JOIN charges c
        ON p.pay_method_id = c.pay_method_id
WHERE c.charge_status = 'Pagado'
```

```
GROUP BY
        p.method_type
;

SELECT *
FROM vw_ingresos_ft_metododepago;
```

TABLAS UTILIZADAS: payment_methods, charges.

#4 Esta view nos ayuda a identificar los clientes que están morosos de pago.

```
CREATE VIEW vw_control_morosos AS
SELECT
        u.first_name as nombre,
   u.last_name as apellido,
   u.email,
   u.document as documento,
   sum(c.total_amount) as total_adeudado,
   c.currency as moneda
FROM users u
INNER JOIN trips t
        ON u.user_id = t.user_id
INNER JOIN charges c
        ON t.trip_id = c.trip_id
WHERE c.charge_status IN ('Pendiente','Fallido')
GROUP BY
        u.user_id,
   u.first_name,
   u.last_name,
   u.email,
   u.document,
   c.currency
;

SELECT *
FROM vw_control_morosos;
```

TABLAS UTILIZADAS: users, charges.

#5 Ranking de heavy users, check de usabilidad y frecuencia.

```
CREATE VIEW vw_ranking_usuarios AS
SELECT
        u.first_name as nombre,
   u.last_name as apellido,
   count(t.trip_id) as cant_viajes,
   sum(t.distance_km) as distancia_total_recorrida,
```

```
        DENSE_RANK() OVER (
                ORDER BY count(t.trip_id) DESC, sum(t.distance_km) DESC
        ) as puesto_ranking
FROM users u
INNER JOIN trips t
        ON u.user_id = t.user_id
GROUP BY
        u.user_id,
   u.first_name,
   u.last_name
;

SELECT *
FROM vw_ranking_usuarios;
```

TABLAS UTILIZADAS: users, trips.

FUNCTIONS

#1 en este caso creamos una función que nos traerá el total de minutos que dura cada viaje, insertando el id del viaje. Se utiliza un cálculo interno sin recurrir a otro cruce de tabla.

```
DELIMITER //

CREATE FUNCTION fn_duracion_viaje_minutos(p_trip_id INT)
RETURNS INT
NOT DETERMINISTIC
READS SQL DATA
BEGIN
        DECLARE v_minutos INT;

   SELECT TIMESTAMPDIFF(MINUTE, start_time, end_time) INTO v_minutos
        FROM trips
        WHERE trip_id = p_trip_id;

   RETURN v_minutos;
END //

DELIMITER ;

SELECT
        trip_id,
        distance_km,
   fn_duracion_viaje_minutos(trip_id) AS duracion_minutos
FROM Trips;
```

TABLAS UTILIZADAS: trips.

#2 En esta función personalizada se consigue el paradero actual de cada scooter que indiquemos a través de su scooter_id. En este caso reemplazamos el join con una subconsulta para poder traernos la estación asignada al scooter.

```
DELIMITER //

CREATE FUNCTION fn_estacion_actual_scooter(p_scooter_id INT)
RETURNS VARCHAR(100)
NOT DETERMINISTIC
READS SQL DATA
BEGIN
	DECLARE v_nombre_estacion VARCHAR(100);

   SELECT name INTO v_nombre_estacion
   FROM stations
   WHERE station_id = (SELECT last_station_id FROM scooters WHERE scooter_id =
p_scooter_id);

   RETURN v_nombre_estacion;
END //

DELIMITER ;

SELECT
	model,
	battery_level,
   fn_estacion_actual_scooter(scooter_id) AS estacion_actual
FROM Scooters;
```

TABLAS UTILIZADAS: stations.

Stored Procedures

#3 En este stored procedure se ejecuta un ordenamiento de tabla, de forma de SQL Dinámico donde asignamos la función a la variable y le damos la instrucción.

```
DELIMITER //
-- ORDENAMIENTO POR CAMPO PREFERENTE DE LA TABLA Y LUEGO POR SENTIDO
DE ORDEN ASC O DESC
 CREATE PROCEDURE sp_ordenar_tb_scooters (
	IN p_campo_de_orden VARCHAR(50),
   IN p_sentido_de_orden VARCHAR(5)
)
BEGIN
```

```
        SET @intruccion_sql = CONCAT('SELECT * FROM scooters ORDER BY ',
p_campo_de_orden,' ', p_sentido_de_orden);

   PREPARE stmt FROM @intruccion_sql;
   EXECUTE stmt;
   DEALLOCATE PREPARE stmt;

END //
DELIMITER ;

CALL sp_ordenar_tb_scooters ('battery_level', 'DESC');
CALL sp_ordenar_tb_scooters ('year', 'ASC');
```

TABLAS UTILIZADAS: scooters.

#4 Stored procedures de SQL estático para agregar y eliminar registros en una tabla. En el ejemplo sería una estación de scooters.

```
DELIMITER //

CREATE PROCEDURE sp_registrar_nueva_estacion(
        IN p_nombre VARCHAR(100),
   IN p_direccion VARCHAR(200),
        IN p_capacidad INT
)
BEGIN
        INSERT INTO stations (name, address, capacity)
   VALUES (p_nombre, p_direccion, p_capacidad);

        SELECT 'Estación registrada correctamente' AS Mensaje;
END //

DELIMITER ;

CALL sp_registrar_nueva_estacion('Estación Puerto', 'Av. Costanera 1000', 35);

select *
from stations;


DELIMITER //

CREATE PROCEDURE sp_eliminar_estacion(
        IN p_station_id INT
)
BEGIN
        DELETE FROM stations
```

```
    WHERE station_id = p_station_id;

        SELECT 'Estación eliminada correctamente' AS Mensaje;
END //

DELIMITER ;

CALL sp_eliminar_estacion (7);

select *
from stations;
```

TABLAS UTILIZADAS: stations.

TRIGGERS

```
CREATE TABLE log_Scooters (
        id_log INT AUTO_INCREMENT PRIMARY KEY,
    scooter_id int,
    accion VARCHAR(10),
    status_anterior VARCHAR(50),
    status_nuevo VARCHAR(50),
    old_value_battery INT,
    new_value_battery INT,
    user_sql_database VARCHAR(100),
    action_date DATETIME
);

#1er Trigger Log de auditoría para cambios/actualizaciones sobre la tabla de scooters

DELIMITER //
CREATE TRIGGER trg_auditoria_cambios_scooters
AFTER UPDATE ON scooters
FOR EACH ROW
BEGIN
        INSERT INTO log_scooters (
                scooter_id,
        accion,
        status_anterior,
        status_nuevo,
        old_value_battery,
        new_value_battery,
        user_sql_database,
        action_date
        )
    VALUES (
                OLD.scooter_id,
        'UPDATE',
```

```
        OLD.scooter_condition,
        NEW.scooter_condition,
        OLD.battery_level,
        NEW.battery_level,
        CURRENT_USER(),
        NOW()
           );
END //
DELIMITER  ;

SELECT * FROM scooters;


UPDATE scooters
SET scooter_condition = 'Batería Baja', battery_level = 15.80
WHERE scooter_id = 1;

SELECT * FROM log_Scooters;

DROP TRIGGER IF EXISTS trg_auditoria_cambios_scooters;
```

TABLAS UTILIZADAS: scooters.

#2do trigger - cambia el estado del scooter cuando inicia un viaje de forma automática.

```
DELIMITER //

CREATE TRIGGER trg_actualizar_condicion_scooter
AFTER INSERT ON trips
FOR EACH ROW
BEGIN

    UPDATE scooters
    SET scooter_condition = 'En Uso'
    WHERE scooter_id = NEW.scooter_id; -- Usamos el ID del scooter del viaje que se acaba
de crear
END //

DELIMITER ;

INSERT INTO trips (user_id, scooter_id, station_start_id, start_time)
VALUES (10, 2, 1, NOW());

SELECT *
FROM scooters
WHERE scooter_id = 2;
```

TABLAS UTILIZADAS: scooters, trips.